Behavior Driven Development(BDD)

BDD stands for Behavior-Driven Development. It is a software development approach that improves collaboration between Business Analysts, Developers, and Testers. In BDD, requirements are written in simple English using the Given-When-Then format, making them easy for everyone to understand. We use Cucumber to implement BDD in Selenium. The feature file contains scenarios, the step definition contains the Java implementation, and the runner class executes the tests.

BDD Structure

BDD scenarios are written using:

- **Given** – Precondition or initial state
- **When** – Action performed by the user
- **Then** – Expected outcome

Advantages of BDD

- Improves communication between business and technical teams.
- Requirements are easy to understand.
- Test cases act as documentation.
- Encourages collaboration.
- Supports test automation.

Common BDD Tools

- Cucumber
- JBehave
- SpecFlow
- Behave (Python)

What is TDD?

TDD stands for Test-Driven Development. It is a software development approach in which developers write test cases before writing the actual application code. It follows the Red-Green-Refactor cycle. First, we write a test case, then write the code to make the test pass, and finally refactor the code if needed. In Java, TDD is commonly implemented using JUnit or TestNG.

What is the TDD cycle?

Answer:

The TDD cycle follows **Red → Green → Refactor**.

- ● **Red** → Write a test case. It fails because the functionality is not implemented.
- □ **Green** → Write the minimum code to make the test pass.
- ◍ **Refactor** → Improve the code without changing its behavior.

What is BDD?

BDD stands for Behavior-Driven Development. It is a software development approach where Business Analysts, Developers, and Testers collaborate to define the application's behavior before development starts. The scenarios are written in simple English using the Given-When-Then format. In Java automation, we use Cucumber to implement BDD.

TDD

Developer wants to create an `add()` method.

1. Write a test:

```
assertEquals(5, add(2,3));
```

2. Test fails.
3. Write:

```
int add(int a, int b){
    return a+b;
}
```

4. Test passes.

➞ Focus is on writing code and testing it.

BDD

For a login feature, the scenario is written first:

```
Given the user is on the login page
When the user enters valid credentials
Then the user should be redirected to the home page
```

Then the automation code is written.

➞ Focus is on application behavior from the user's perspective.

TDD (Test-Driven Development) vs BDD (Behavior-Driven Development)

TDD	BDD
TDD focuses on testing the code and its functionality.	**BDD focuses on testing the application's behavior based on business requirements.**
Test cases are written **before** the actual code.	Scenarios are written **before** development starts.

TDD	BDD
Tests are written in a technical format, so they are mainly understood by developers.	Scenarios are written in simple English using **Given-When-Then**, so developers, testers, and business stakeholders can understand them.
Mainly used by developers.	Used by developers, testers, and business stakeholders.
Verifies whether the code works correctly.	Verifies whether the application behaves as expected.
Commonly used for Unit Testing. ✅	**Commonly used for Acceptance Testing and Functional Testing.** ✅
Common tools: **JUnit**, **TestNG**	Common tools: **Cucumber**, **SpecFlow**

In TDD, developers write test cases before writing the application code. The main goal is to verify that the code works correctly. It is mainly used by developers for unit testing, using frameworks like **JUnit** or **TestNG.**

In BDD, developers, testers, and business stakeholders first define the application's expected behavior using the Given-When-Then format. The goal is to ensure the application behaves according to business requirements. It is commonly used for acceptance and functional testing, using **Cucumber.**

The main difference is that TDD focuses on testing the code, whereas BDD focuses on testing the application's behavior according to business requirements."

Memory Trick

TDD = Test the Code

BDD = Test the Behavior ✅

What is Cucumber?

Cucumber is a BDD (Behavior-Driven Development) tool used for test automation. It allows us to write test scenarios in simple English using the Gherkin language, so that developers, testers, and business stakeholders can easily understand the requirements.

Cucumber reads the feature file, maps each step to the Step Definition, and executes the tests using Selenium. It is mainly used for functional and acceptance testing.

The main components of Cucumber are:

1. Feature File – Contains test scenarios written in Gherkin language using keywords like Feature, Scenario, Given, When, and Then.

2. Step Definition – A Java class that contains the implementation of the steps written in the feature file. It connects the feature file with Selenium code.

3. Runner Class – Used to execute the feature files. It tells Cucumber which feature files and step definitions to run.

4. Hooks – Used to execute setup and cleanup code before and after a scenario, such as opening and closing the browser.

Requirement

↓

Feature File (.feature)

↓

Step Definition (Java)

↓

Runner Class

↓

Selenium

↓

Browser Execution

What is the role of Cucumber?

Cucumber itself does not automate the browser. Its role is to read the feature file, match each step with the Step Definition, and trigger Selenium to execute the browser actions**.**

Given, When, Then in Cucumber

Given, When, and Then are Gherkin keywords used in Cucumber to describe the behavior of an application in a readable format.

- Given represents the precondition or initial state of the application.
- When represents the action performed by the user.
- Then represents the expected result after the action is performed.

- **Given** → Precondition (initial setup)
- **When** → Action performed by the user
- **Then** → Expected result

Example

Cucumber uses **Gherkin language**:

```
Feature: Login Functionality

Scenario: Successful Login

Given the user is on the login page
When the user enters valid credentials
Then the user should be logged in successfully
```

Why do we use Cucumber?

- Supports BDD approach.
- Scenarios are easy to read and understand.
- Improves communication between technical and non-technical teams.
- Can be integrated with Selenium for automation testing.
- Acts as both documentation and test cases.

Notes

"We use **`System.setProperty()`** to tell Selenium where the browser driver (like ChromeDriver or GeckoDriver) is located." To establish communication between the test script and the browser."

System.setProperty("webdriver.chrome.driver", "C:\\Drivers\\chromedriver.exe");

WebDriver driver = new ChromeDriver(); driver.get("https://www.google.com");

Test Runner class

```
@RunWith(Cucumber.class) @CucumberOptions(
features = "src/test/resources/features", glue
= "stepDefinitions",
plugin = {"pretty","html:target/cucumber-report.html"}, monochrome
= true
)
```

- **@RunWith(Cucumber.class)** → Tells JUnit to run Cucumber tests.
- **features** → Path of feature files.
- **glue** → Package containing Step Definition files.
- **plugin** → Generates reports.
- **monochrome = true** → Gives clean console output.

Plugin can be any of these

`html:target/report.html` Generates HTML report.

`json:target/report.json` Generates JSON report.

`junit:target/report.xml` Generates XML report for Jenkins/CI tools.

Parameterization in Cucumber

Parameterization in Cucumber means passing dynamic test data to Step Definitions instead of hardcoding values. It allows us to run the same test scenario with different sets of test data, improving reusability, reducing code duplication, and supporting data-driven testing.

We can achieve parameterization using:

- **Scenario Outline** with an **Examples** table (most commonly used).
- **Data Tables** for passing multiple rows and columns of data.
- **Doc Strings** for passing large text data.

If the interviewer asks for an example:

For example, if I want to test the login functionality with different usernames and passwords, instead of writing separate scenarios for each user, I use a Scenario Outline and provide multiple test data sets in the Examples table. Cucumber automatically executes the same scenario for each data set.

Data-Driven Testing (DDT) is a testing approach in which same test case is executed multiple times with different sets of input data.

Instead of hardcoding test data in the script, the data is stored in external sources such as:

- Excel
- CSV
- Database
- JSON
- XML

Ways to Achieve Parameterization in Cucumber

1. Using Placeholders (`{string}`, `{int}`)

Feature File

```
Given User enters username "admin"
```

Step Definition

```
@Given("User enters username {string}")
 public void enterUsername(String username) {
    System.out.println(username);
}
```

2.Using Scenario Outline and Examples

Feature File

```
Scenario Outline: Login Test

Given User enters "<username>" and "<password>"

Examples:
| username | password |
| admin    | admin123 |
| user1    | user123  |
```

The same scenario runs multiple times with different data.

Tags in Cucumber

Tags in Cucumber are used to group and categorize test scenarios so that we can execute them selectively.

Instead of running the entire test suite, we can run only the required scenarios, such as Smoke, Regression, or Sanity. This reduces execution time and makes test execution easier to manage.

Important Points

- ✓ Features and Scenarios can be marked with Tags.
- ✓ Tags start with the `@` symbol.
  - Example: `@Smoke`, `@Regression`, `@Sanity`
- ✓ A Feature or Scenario can have one or more tags.

Example:

```
@Smoke @Regression
Feature: Verify Login
```

Running Tags in Runner Class

Single Tag

```
@CucumberOptions(tags = "@Smoke")
```

Multiple Tags (OR condition)

```
@CucumberOptions(tags = "@Smoke or @Regression")
```

Executes scenarios that have **either** `@Smoke` **or** `@Regression`.

AND condition

```
@CucumberOptions(tags = "@Smoke and @Regression")
```

Executes only scenarios that have **both** tags.

NOT condition

```
@CucumberOptions(tags = "not @Regression")
```

Executes all scenarios **except** those tagged with `@Regression`.

Where can we place Tags?

✅ Tags can be placed above:

- **Feature**
- **Scenario**
- **Scenario Outline**
- **Examples**

❌ Tags **cannot** be placed above:

- **Background**
- **Given**
- **When**
- **Then**
- **And**
- **But**

Tag Inheritance (Very Important)

If you place a tag above a **Feature**, all the Scenarios, Scenario Outlines, and Examples inside that Feature automatically inherit that tag.

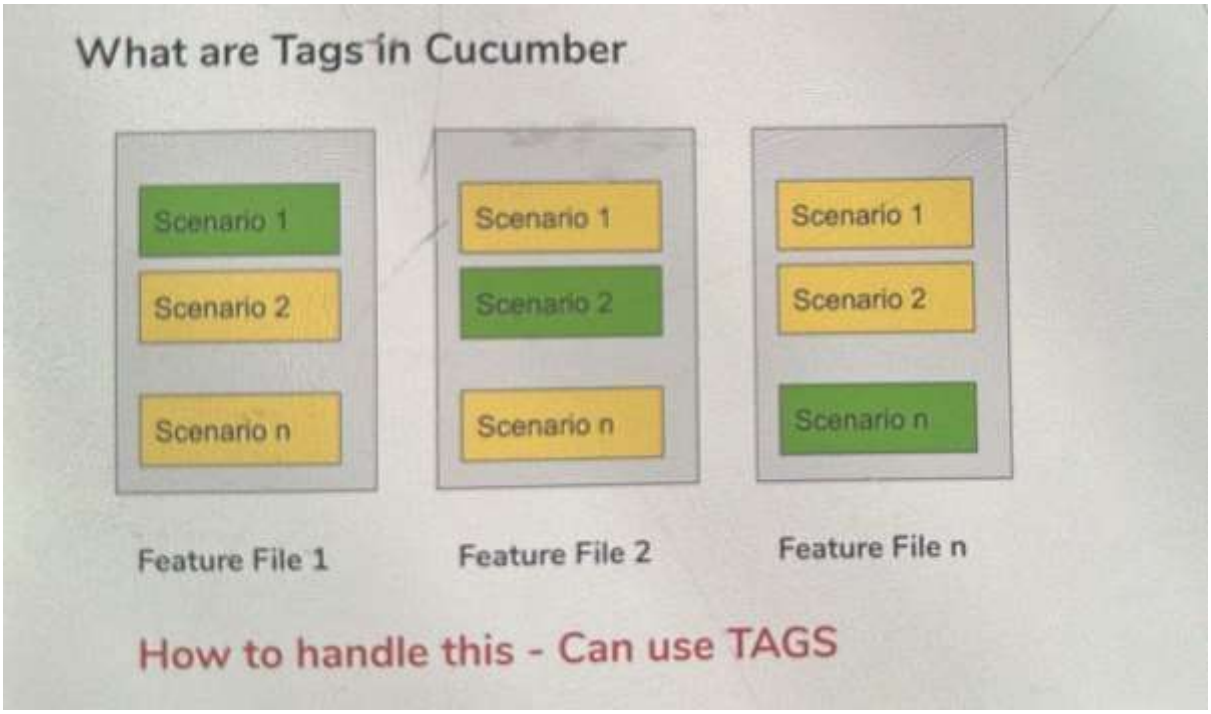
Example:

```
@Smoke
Feature: Login

Scenario: Valid Login

Scenario: Invalid Login
```

👉 Both scenarios are treated as `@Smoke` even though you didn't write `@Smoke` above each scenario.



Interview Answer

Tags in Cucumber are used to group and categorize test scenarios so that we can execute them selectively.They start with the `@` symbol, such as **`@Smoke`**, **`@Regression`**, or **`@Sanity`**. We can run a single tag, multiple tags using AND or OR conditions, or exclude tags using the NOT condition. Tags can be placed above a Feature, Scenario, Scenario Outline, or Examples, but they cannot be placed above a Background or individual steps. If a tag is placed above a Feature, it is automatically inherited by all the scenarios within that Feature.

Hooks

Hooks are special annotations that execute automatically before or after the execution of a scenario.

They are used to perform common setup and cleanup activities without writing the same code in

every Step Definition.

The most commonly used Hook annotations are:

- **`@Before`** – Executes before every scenario and is used for setup activities like launching the browser or initializing the WebDriver.
- **`@After`** – Executes after every scenario and is used for cleanup activities like closing the browser, taking screenshots on failure, or releasing resources.

Interview Answer

Hooks in Cucumber are special annotations used to execute methods automatically before or after every scenario. They are mainly used for common setup and cleanup activities, such as launching the browser, initializing the WebDriver, closing the browser, taking screenshots on failure, and releasing resources. The `@Before` annotation executes before every scenario, while

the `@After` annotation executes after every scenario. Hooks help reduce code duplication and improve the maintainability of the automation framework.

Order in Hooks

When multiple hooks of the same type are present, **Order** is used to control the execution sequence.

Example

```
@Before(order = 1) public
void setupDB() {
    System.out.println("Database Connection");
}

@Before(order = 2)
public void launchBrowser() {
    System.out.println("Browser Launch");
}
```

For `@Before` hooks, lower order value executes first.

For `@After` hooks, higher order value executes first (reverse order).

Conditional Hooks

Conditional hooks are the hooks that execute only for scenarios with specific tags

Instead of running before/after every scenario, they run only for tagged scenarios.

Example

```
@Before("@Smoke")
 public  void  setupSmoke()
{
    System.out.println("Executing Before Hook for Smoke Tests"); }
```

This hook runs only for scenarios tagged with **@Smoke**.

Feature File

```
@Smoke
Scenario: Login Test
Given User is on Login Page

@Regression
Scenario: Add Product Test
Given User is on Product Page
```

Execution

✅ Login Test → Hook executes

❌ Add Product Test → Hook does not execute

How to Use both tags and order

```
@Before(value = "@Smoke", order = 1) public
void setupDB() {
System.out.println("Database Connection");
}
```

Background in Cucumber

Background is a Gherkin keyword is used to define a set of common steps that should be executed before every scenario in a feature file.

It is mainly used to avoid repeating common precondition steps, improve readability, and reduce duplication in the feature file.

only one Background section is allowed per feature file.

Why do we use Background?

- To avoid repeating common steps in every scenario.
- To improve readability and maintainability.
- To reduce duplication in feature files.
- Unlike hooks,background is visible to the readers of the feature file

Without Background

```
Scenario: Valid Login
Given User is on Login Page
When User enters valid credentials
Then Login should be successful

Scenario: Invalid Login
Given User is on Login Page
When User enters invalid credentials
Then Error message should be displayed
```

Here, **"Given User is on Login Page"** is repeated.

With Background

```
Feature: Login Functionality

Background:
Given User is on Login Page
```

```
Scenario: Valid Login
When User enters valid credentials
Then Login should be successful

Scenario: Invalid Login
When User enters invalid credentials
Then Error message should be displayed
```
Now

the common step is written only once.

Execution Flow

```
Background
↓
Scenario 1

Background
    ↓
Scenario 2

Background
↓
Scenario 3
```

Background executes before every scenario.

U can only have one set of Background steps per feature

Can you explain with an example?

Yes. For example, suppose I have multiple login scenarios. Every scenario starts with the step "Given the user is on the Login page". Instead of writing this step in every scenario, I write it once in the Background section. Then Cucumber automatically executes the Background steps before every scenario.

Background vs Hooks

Background	Hooks
Written in **Feature File**.	Written **Java Step Definition/Hook Class**.
Used for **common preconditions**.	Used for **setup and teardown activities**.
Background	**Hooks**
Visible to business users.	Not visible in the feature file.
Written using Gherkin steps (Given, When, Then).	Written using annotations like `@Before`, `@After`.

Executes before every scenario in that scenarios).	Executes before/after every scenario (or tagged feature file.
Example: User is on Login Page.	Example: Launch Browser, Close Browser, DB Connection.

Interview Answer

Background is used for common business steps that are shared by all scenarios, such as navigating to the login page. Hooks are used for technical setup and cleanup activities, such as launching the browser or closing the browser. Background is written in the Feature file using the **`Background`** keyword, whereas Hooks are written in a Java class using **`@Before`** and **`@After`** annotations.

Command Line Excecution

Command Line Execution means running tests or features from command line or terminal without using the IDE or GUI.

Why do we use Command Line Execution?

- Execute tests without IDE.
- Useful in Jenkins and CI/CD pipelines. Faster automation execution.
- Easy integration with Maven

When to Use Command Line Execution?

- Whenever you need to run the tests **without opening the IDE**.
- Whenever you need to do **integration with other processes like CI/CD and DevOps**.
- Whenever you are using **CI tools like Jenkins**.
- Whenever you need **batch execution or scheduled execution**.
- Whenever you are done with your **test creation and test setup**.

How to Perform Command Line Execution?

Step 1: Open Command Prompt / Terminal

Navigate to your project folder:

```
cd C:\Users\vara\AutomationProject
```

Step 2: Verify Maven Installation

```
mvn -version
```

Step 3: Execute All Tests

```
mvn test
```

Step 4: Execute Specific Tagged Tests

Run Smoke Tests: `mvn test -Dcucumber.filter.tags="@Smoke"`

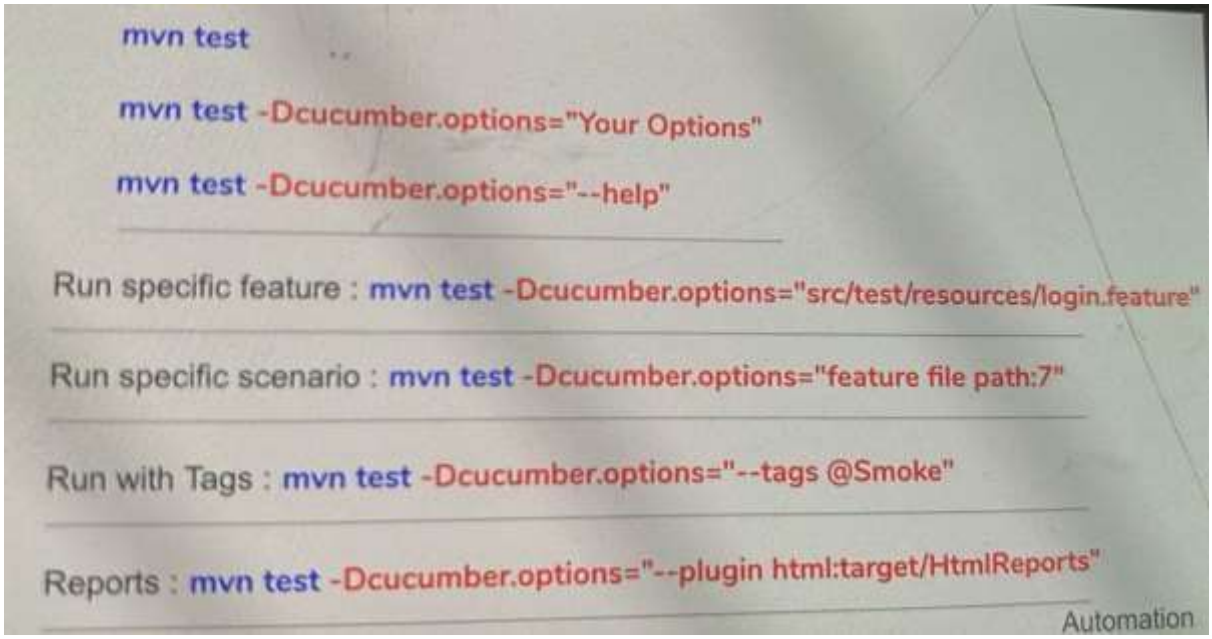
Run Regression Tests: `mvn test -Dcucumber.filter.tags="@Regression"` Run Multiple Tags: `mvn test -Dcucumber.filter.tags="@Smoke or @Regression"`

Step 5: Clean and Execute

```
mvn clean test
```

- `clean` → Removes previous build files.
- `test` → Executes test cases.

Old versions



Latest Versions

To Run a Specific Feature File mvn test -Dcucumber.features="src/test/resources/features/Login.feature" **Run Specific Scenario Using Line Number**

```
mvn test -Dcucumber.options="src/test/resources/features/Login.feature:7"
```

- `Login.feature` → Feature file path.
- `:7` → Line number where the scenario starts

(or)

Using tag: `mvn test -Dcucumber.filter.tags="@LoginTest"`

Run with tags

```
mvn test -Dcucumber.filter.tags="@Smoke"
```

Run Multiple Tags

```
mvn test -Dcucumber.filter.tags="@Smoke or @Regression"
```

Generate Reports HTML

Report

`mvn test -Dcucumber.plugin="html:target/cucumber-report.html` **JSON**

Report

```
mvn test -Dcucumber.plugin="json:target/report.json"
```

Unit XML Report

```
mvn test -Dcucumber.plugin="junit:target/report.xml"
```

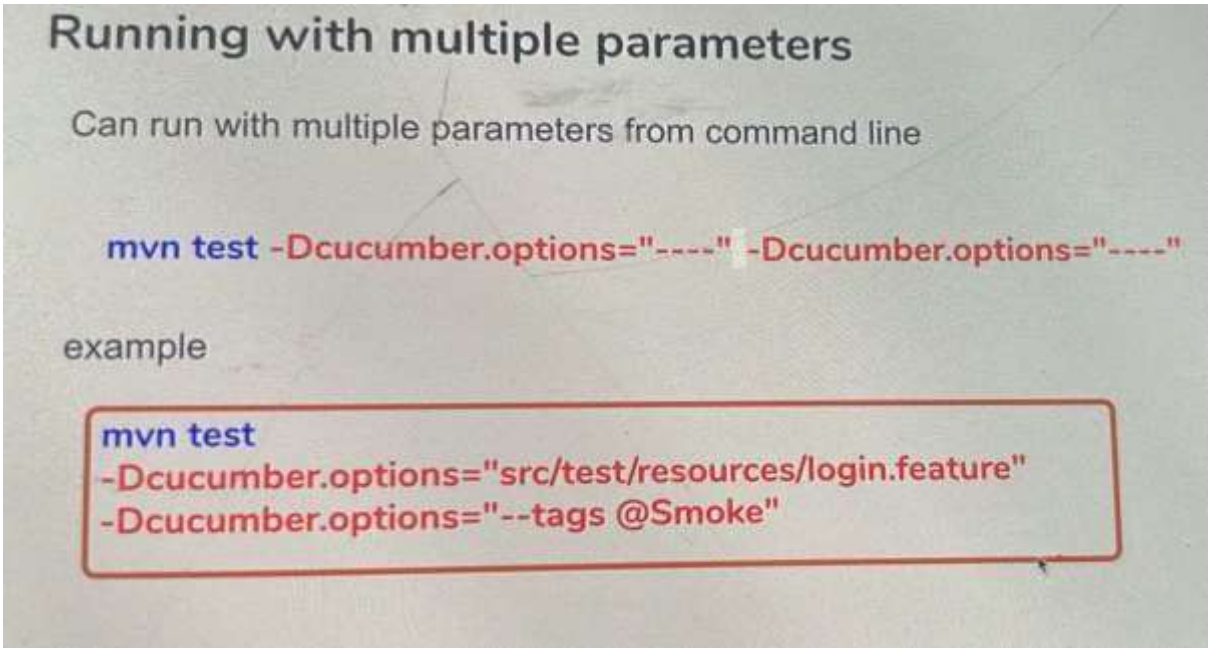
Pretty Console Output

`mvn test -Dcucumber.plugin="pretty"` **Multiple**

Plugins

```
mvn test -
Dcucumber.plugin="pretty,html:target/report.html,json:target/report.json"
```

In old version



In new version

Run Specific Feature + Tags

mvn test -Dcucumber.features="src/test/resources/login.feature" -Dcucumber.filter.tags="@Smoke or @Regression"

By default mvn test will run the files with naming syntax(Test Runner file)

**/Test*.java

**/*Test.java

**/*TestCase.java

Reports

"Reports in Cucumber are used to capture and display test execution results in different formats such as HTML, JSON, XML, and Console output."

Why do we use Reports?

- To view test execution results.
- To identify passed, failed, and skipped scenarios.
- To share execution results with stakeholders.
- To integrate with CI/CD tools like Jenkins.
- To analyze test execution history.

Dry Run

@CucumberOptions(dryRun = true)

"Dry Run is used to check whether all feature file steps have corresponding step definitions without executing the tests."

Interview Questions and answers

What is Gherkin?

Answer:
Gherkin is the language used to write Cucumber feature files. It uses keywords such as Feature, Scenario, Given, When, Then, And, and But.

Why do we use Cucumber?

"Cucumber improves collaboration between developers, testers, and business teams by allowing test cases to be written in a readable format." **What are Plugins?**

"Plugins are used to generate execution reports and customize output."

Examples:

- pretty
- html
- json
- junit

Why Selenium + Cucumber?

"Selenium is used for browser automation, while Cucumber provides a BDD framework to write test scenarios in a readable format. Together they enable business-readable automation testing."

What is a Feature File?

A Feature File contains multiple scenarios related to a particular feature and is written in Gherkin language with a `.feature` extension." **.**

What is a Scenario?

A Scenario represents a single test case that verifies a specific functionality of the application. It consists of a sequence of steps written using Gherkin keywords such as Given, When, Then, And, and But."

What is Scenario Outline?

Answer:

Scenario Outline is used to execute the same scenario multiple times with different sets of test data using an Examples table.

Difference Between Scenario and Scenario Outline?

Scenario

- Executes once.
- Uses one set of data.

Scenario Outline

- Executes multiple times.
- Uses multiple sets of data through Examples table.

. What is a Step Definition?

Answer:
A Step Definition contains Java code that implements the steps written in the feature file.

What is a Test Runner Class?

Answer:
A Test Runner Class is used to execute Cucumber feature files and configure options such as features, glue, tags, and plugins.

What is Glue?

Answer:
Glue specifies the package location of Step Definition and Hook classes.

```
glue="stepDefinitions"
```

How Do You Run a Specific Scenario?

Answer: Using tag: `mvn test -Dcucumber.filter.tags="@LoginTest"` Using line number (older versions): `mvn test -Dcucumber.options="Login.feature:10"`

Difference Between Background and Scenario Outline

Background

- Common steps before every scenario.

Scenario Outline

- Same scenario with multiple data sets.

Cucumber Architecture

Feature File
↓
Step Definition
↓
Runner Class
↓
Selenium WebDriver
↓
Application

Difference Between And and But

And

- Additional positive condition.

But

- Additional negative condition.

What is Monochrome in Cucumber?

"Monochrome is an option in Cucumber that makes the console output more readable and cleaner by removing unnecessary special characters and formatting."

Syntax

```
@CucumberOptions(
monochrome = true )
```

What is data table?

A Data Table is used to pass multiple rows and columns of test data from the Feature file to the Step Definition in a tabular format. It is mainly used when a step requires multiple input values, making the test data easy to read, maintain, and reuse.

Data Table = Multiple rows + Multiple columns + One Step."

When I enter the following employee details

Name	Age	City
Ram	25	Hyd
Ravi	30	Blr

```
@When("I enter the following credentials")
public void enterCredentials(DataTable dataTable) {

List<Map<String, String>> data = dataTable.asMaps();

for (Map<String, String> row : data) {
String username = row.get("username");
String password = row.get("password");

System.out.println(username + " " + password);
}
}
```

Cucumber reads the Data Table from the Feature file and passes it to the Step Definition as a `DataTable` object. We then convert it into a Java collection, such as a List or Map, to access and process the test data based on our automation requirements.

: What is the difference between Scenario Outline and Data Table?

Scenario Outline and Data Table are both used for parameterization in Cucumber, but they are used in different situations.

Scenario Outline is used when we want to execute the **same scenario multiple times** with different sets of test data. It uses an **Examples** table, and Cucumber executes the entire scenario once for each row in the Examples table.

Data Table is used when we want to pass **multiple rows and columns of data to a single step** in a scenario. The entire table is passed to the Step Definition at once, and the scenario executes only once.

What is the difference between Feature and Scenario?

Answer:

A Feature represents a functionality, whereas a Scenario represents a single test case for that functionality. One Feature can contain multiple Scenarios.

Requirement	Command
Run all tests	`mvn test`
Clean + Run	`mvn clean test`
Run Feature file	`mvn test -Dcucumber.features="src/test/resources/features/Login.feature"`
Run Test Runner	`mvn -Dtest=TestRunner test`
Run Smoke	`mvn test -Dcucumber.filter.tags="@Smoke"`
Run Regression	`mvn test -Dcucumber.filter.tags="@Regression"`
Run Feature + Tag	`mvn test -Dcucumber.features="src/test/resources/features/Login.feature" -Dcucumber.filter.tags="@Smoke"`